



Programming languages

Lesweek 4

Document : 04 3

rexroth
A Bosch Company

IndraMotion MLC Programming languages

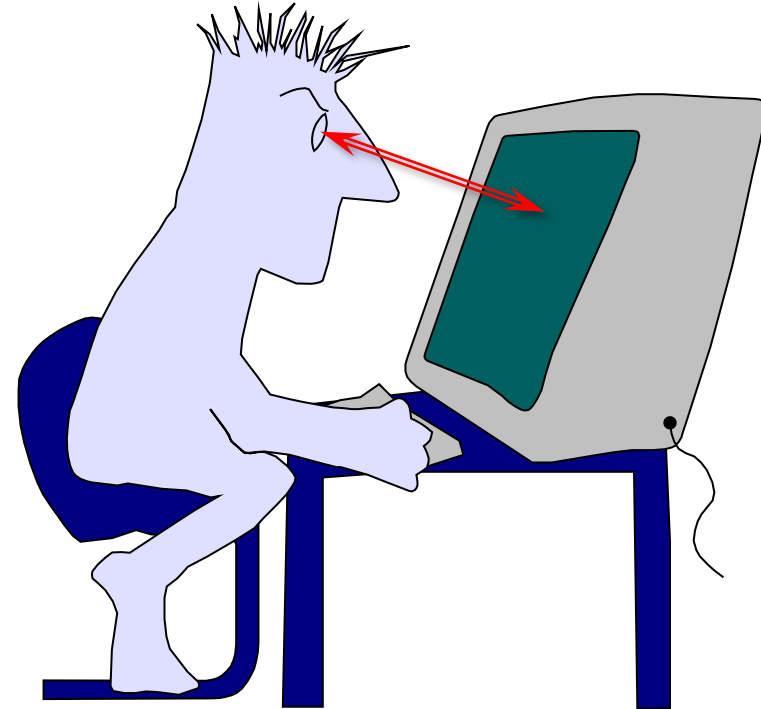
What about IEC 61131?

- The international standard IEC 61131 was created as a basis for uniform PLC programming, where modern software technology designs are considered
- The standard comprises 7 parts:
 - IEC 61131-1 General overview, definitions
 - IEC 61131-2 Hardware
 - IEC 61131-3 Programming languages
 - IEC 61131-4 User guidelines
 - IEC 61131-5 Messaging service specification
 - IEC 61131-7 Fuzzy logic
 - IEC 61131-8 Technical report
- MLC is a IEC 61131 compliant system, i.e. Motion programming is based on this standard
- All IEC programming languages are supported

IndraMotion MLC Programming languages

What about IEC 61131?

- IEC 61131-3 ...
standardizes the way people work with
PLC 's and other industrial controls
- IEC 61131-3 ...
defines the interface between the
programmer and the control system



IndraMotion MLC Programming languages

Contents of IEC 61131-3

Common elements and concepts

Data types

Variables

Configuration

Resource

Task

Function

Function block

Program

IEC 61131-3

Programming Languages

ST Structured Text

LD Ladder Diagram

~~IL Instruction List~~

FBD Function Block Diagram

SFC Sequential Function Chart

IndraMotion MLC Programming languages

Basic data types

Binary Bit string	Signed integers	Unsigned integers	Floating point	Time, Duration, Date, Character string
BOOL BYTE WORD DWORD LWORD	SINT INT DINT LINT	USINT UINT UDINT ULINT	REAL LREAL	TIME DATE TIME_OF_DAY or TOD DATE_AND_TIME or DT STRING

Meaning of 1st letter:

D = double (32 Bit), L = long (64 Bit), S = short (8 Bit), U = unsigned

Bool = 1Bit, Byte = 8 Bit, Word = 16 Bit.

IndraMotion MLC Programming languages

Own data types / Type definitions

- Definition of own data types based on the basic data types:
 - Definition of customized initial values
 - Enumerators
 - Types with restricted range of values
 - Arrays
 - Structures

IndraMotion MLC Programming languages

Examples for Type definitions

TYPE

```
usiOverride : USINT;                                (* own type designation *)

usiOverride50 : USINT := 50;                         (* with initial value "50"*)

usiSpindleOverride : USINT (20 .. 150);              (* restricted range of values *)

enLanguage : (German, English, French, Italian);     (* Enumerator *)

arOverrides : ARRAY[0 .. 3] OF tOverride;            (* one-dimensional array *)

stBook :
STRUCT
    Title      : STRING(255);
    Author     : STRING (255);
    Language    : eLanguage;
    ISBN       : ULINT;
    OutOfPrint  : BOOL;
END_STRUCT;
END_TYPE
```



(* Structure *)

IndraMotion MLC Programming languages

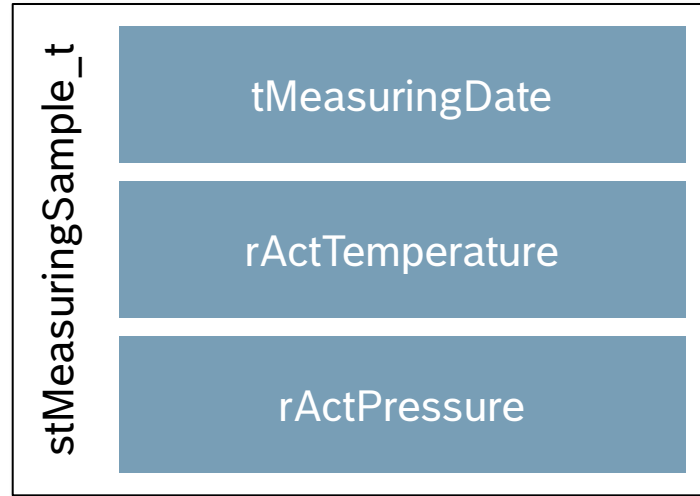
General data types

- Definition of generalized data types to group basic data types
- ... e.g. for standard functions, which can be used for more than one basic data type.

ANY					
ANY_BIT	ANY_NUM			ANY_DATE	TIME STRING Derived data types
BOOL BYTE WORD DWORD LWORD	ANY_INT		ANY_REAL	DATE TIME_OF_DAY or TOD DATE_AND_TIME or DT	
	INT	UINT	REAL LREAL		
	SINT	USINT			
	DINT	UDINT			
	LINT	ULINT			

IndraMotion MLC Programming languages

Structures



TYPE

stMeasuringSample_t:

STRUCT

```
tMeasuringDate      : TIME;  (* time stamp *)
rActTemperature      : REAL;  (* temperature in °C *)
rActPressure         : REAL;  (* pressure in bar *)
```

END_STRUCT;

END_TYPE

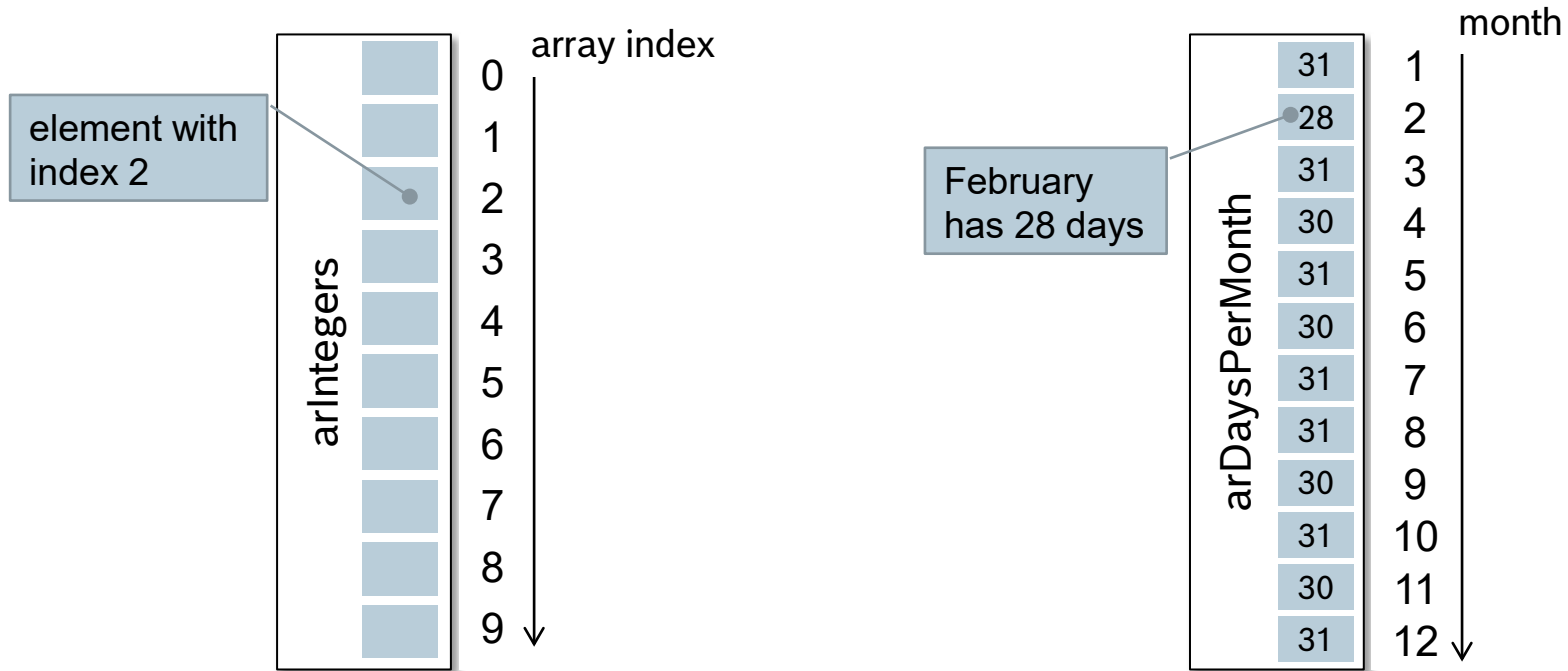
IndraMotion MLC Programming languages

Arrays

VAR

```
arIntegers      : ARRAY[0..9] OF INT;  
arDaysPerMonth  : ARRAY[1..12] OF UINT;
```

END_VAR



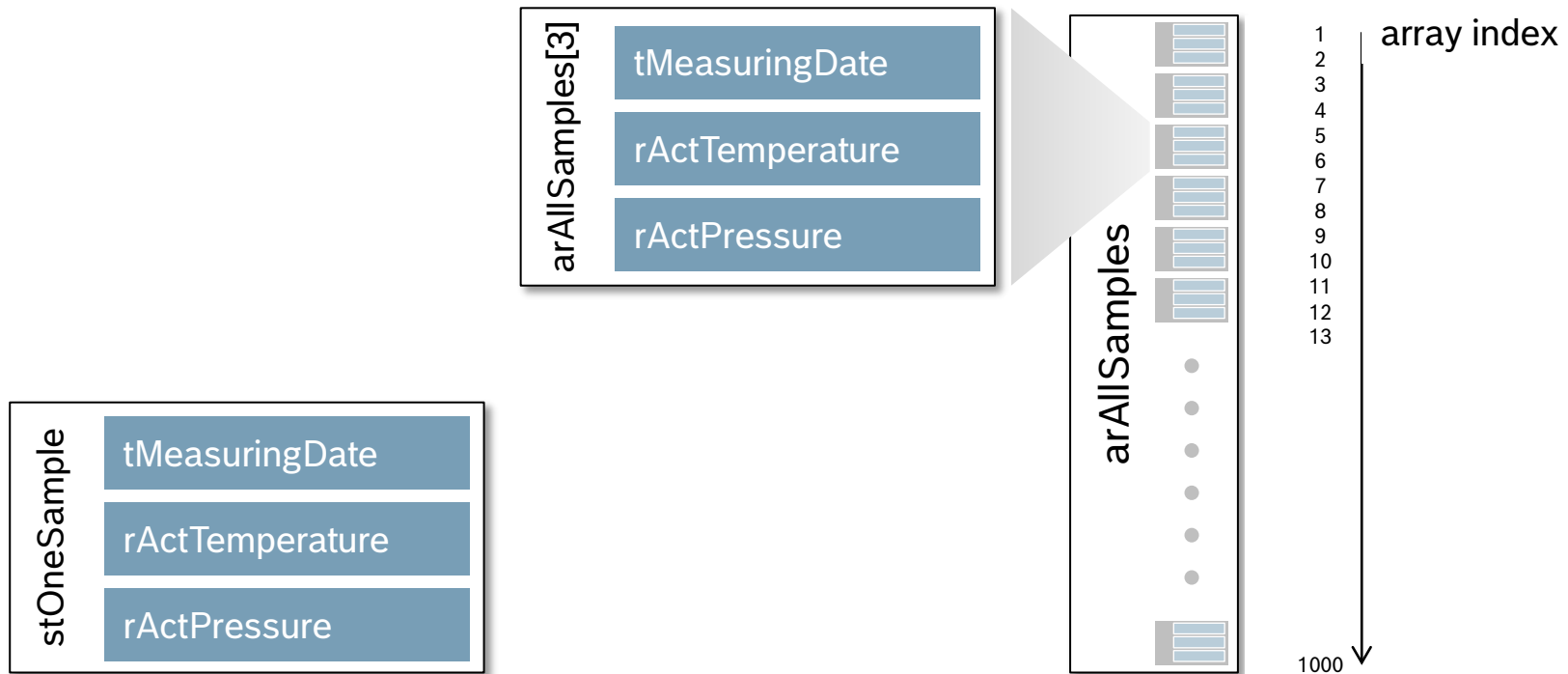
IndraMotion MLC Programming languages

Arrays

VAR

```
stOneSample      : stMeasuringSample_t;  
arAllSamples     : ARRAY[1..1000] OF stMeasuringSample_t;
```

END_VAR



IndraMotion MLC Programming languages

Inputs, outputs and markers

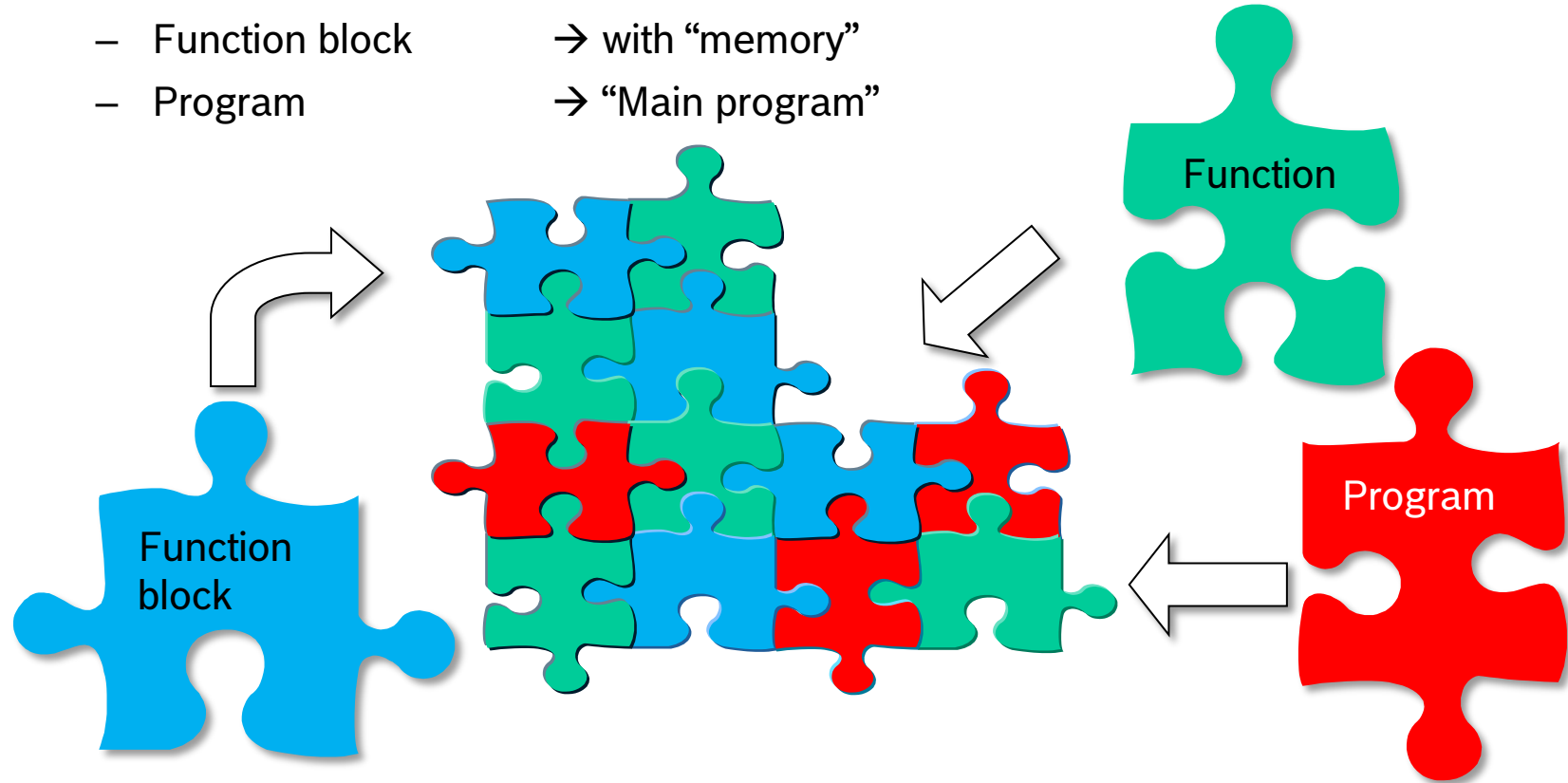
Direct PLC addresses				Remark
%				1 st character
	I Q M			Input Output Marker
		none X B W D L		Bit Bit Byte Word Double word Long word
			v.w.x.y.z	Hierarchical address (with DCC only two-digit)

Examples:	%IW9	Input word 9
	%IX2.5	Input bit 5 in 3 rd input byte
	%QD22	Output double word 22
	%MB9	Marker byte 9

IndraMotion MLC Programming languages

POUs: Program organization units

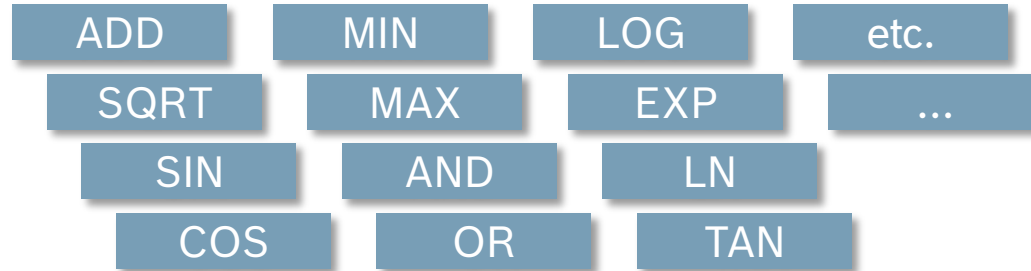
- POUs are used to modularize the overall programming task
 - Function → returns function value based on the input parameters
 - Function block → with “memory”
 - Program → “Main program”



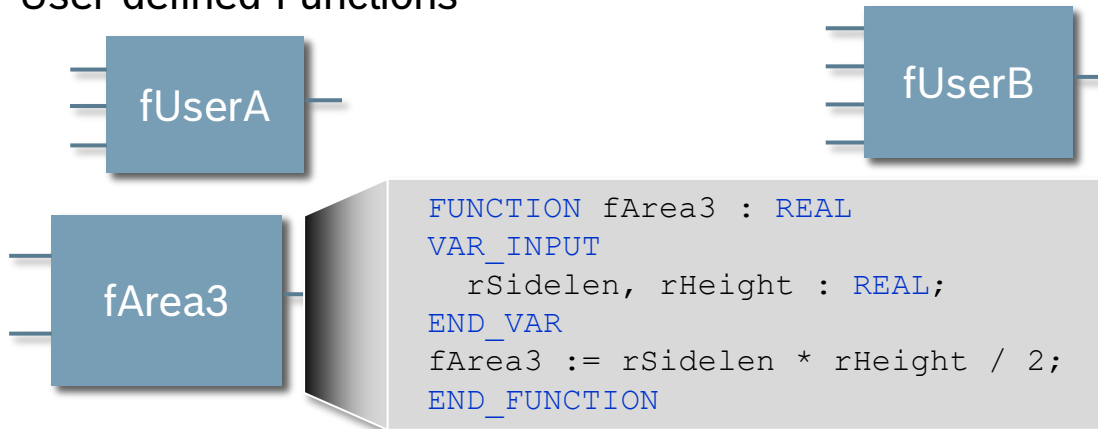
IndraMotion MLC Programming languages

Functions

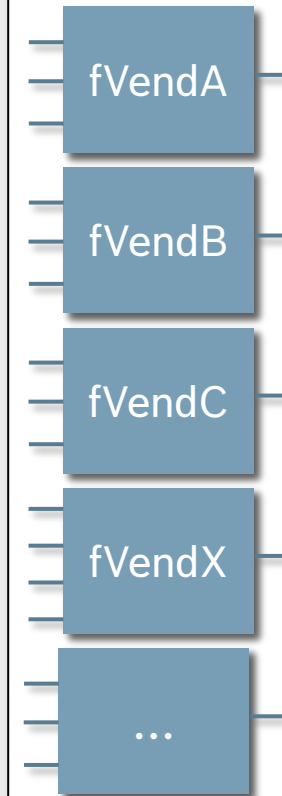
Standard Functions



User-defined Functions



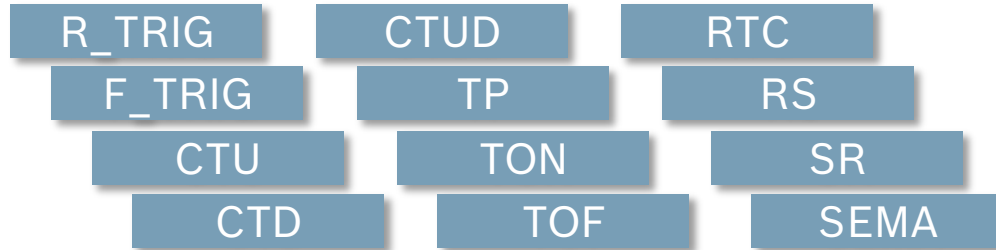
Vendor-specific Functions



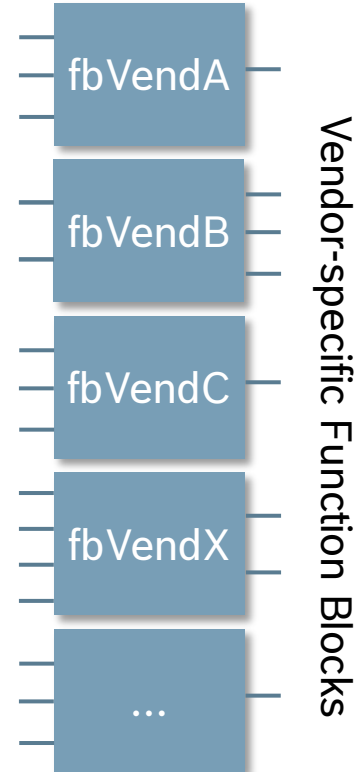
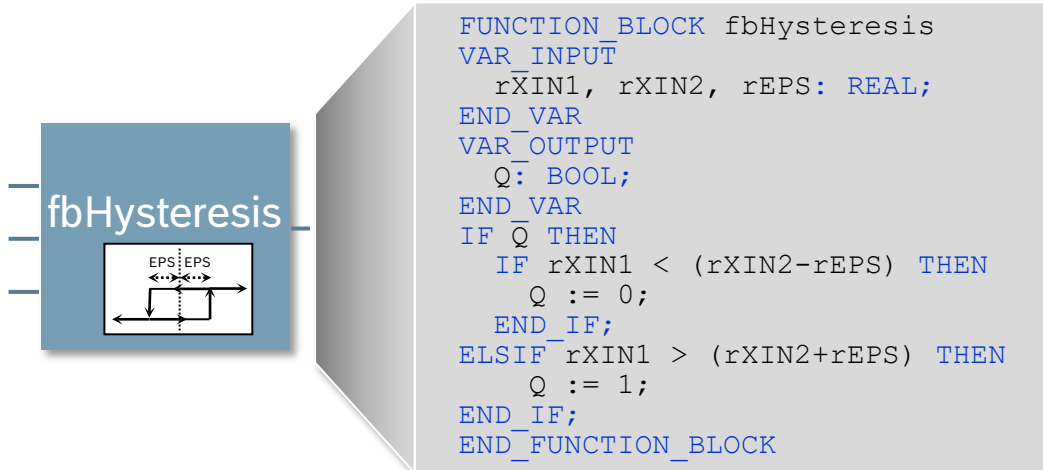
IndraMotion MLC Programming languages

Function Blocks

Standard Function Blocks



User-defined Function Blocks



IndraMotion MLC Programming languages

Functions, Function Blocks and Programs

Function

- Standard Functions
ADD, SQRT, SIN, COS, GT, MIN, MAX, AND, OR ...
- Only implementation of algorithms (code)

Function block

- Standard Function blocks for Timer, Counter, Trigger etc.
- Comparable with IC, i.e., Black Box executing a certain function
- Implementation of code and data (memory)

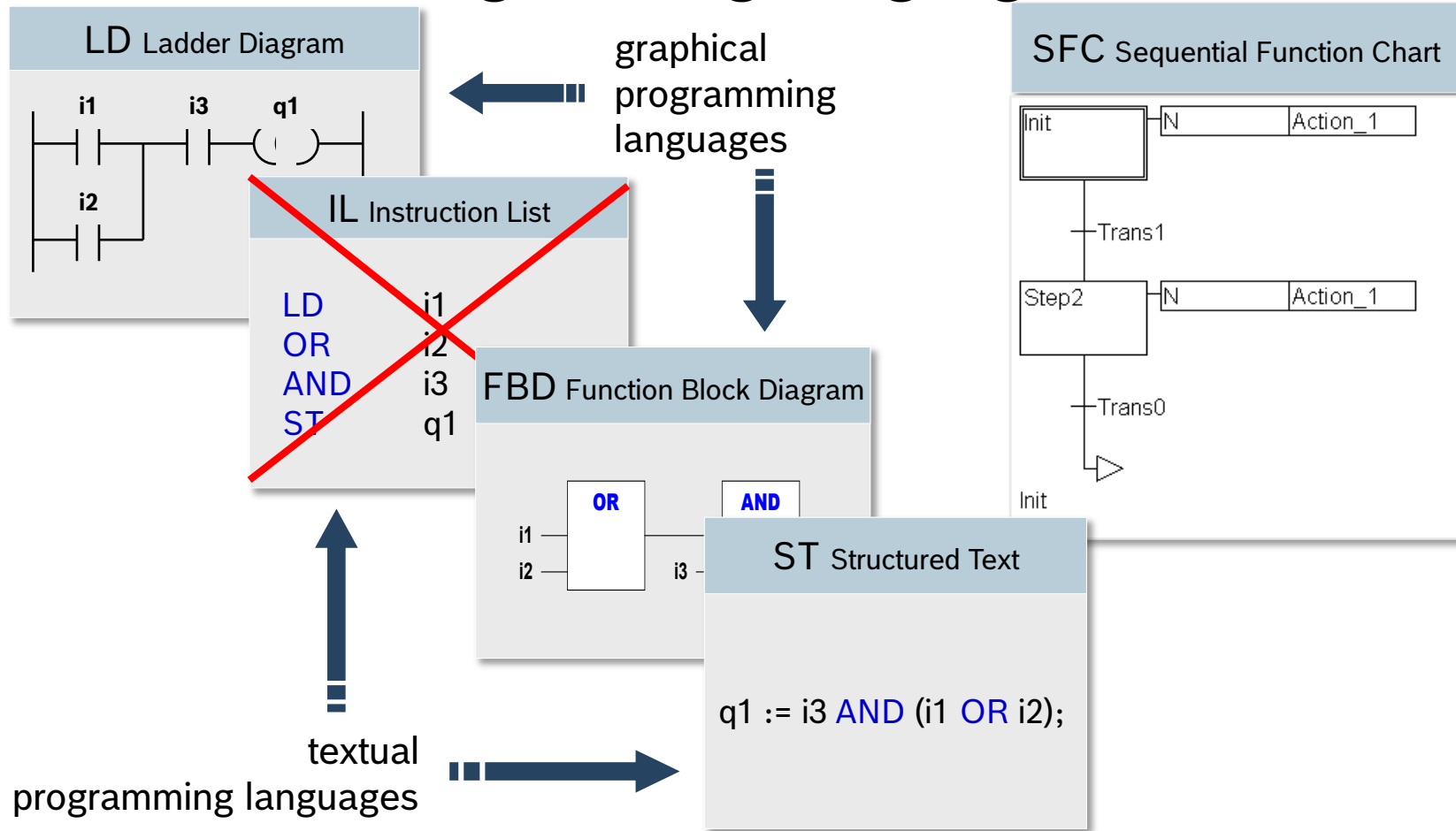
Program

- "Main program"
- Based on Functions and Function blocks
- Only one Instance (unlike FBs which can be instantiated several times)

☞ Implementation in any IEC language (in all cases)

IndraMotion MLC Programming languages

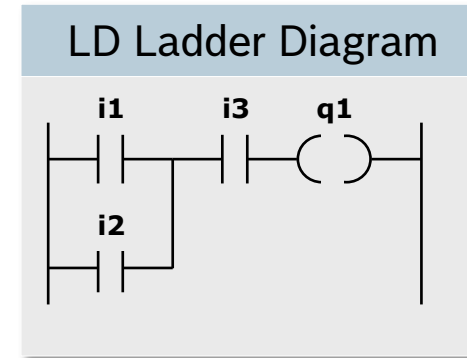
IEC 61131 – Programming Languages



IndraMotion MLC Programming languages

LD Ladder Diagram

- Graphical language
- Available nearly for all classical PLCs
- Standardized rationalized set of relay ladder programming symbols
- Resembles an electrical drawing
- Suited for Boolean logic
- Well-known US-style of programming



IndraMotion MLC Programming languages

IL Instruction List

- Textual language
- Resembles Assembler code (like S5)
- Line oriented:
only one operation allowed per line,
such as storing a value in a register
- Single Accumulator based execution
model
- Based upon the German AWL
(Anweisungsliste)

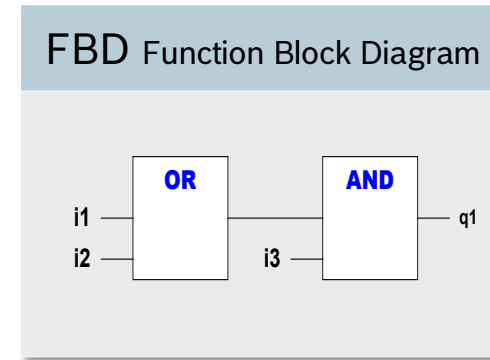
IL Instruction List

LD	i1
OR	i2
AND	i3
ST	q1

IndraMotion MLC Programming languages

FBD Function Block Diagram

- Graphical language
- Allows program elements which appear as blocks to be “wired” together in a form analogous to a circuit diagram
- Easy to understand
- Widely used in Europe



IndraMotion MLC Programming languages

ST Structured Text

- High level language
- Block structured
- Syntax resembles PASCAL
- Complex statements and nested instructions possible
- Constructs to control of the execution path:
 - Iteration loops
(FOR, REPEAT-UNTIL, WHILE-DO)
 - Conditional execution
(IF-THEN-ELSE, CASE)
- Best suited programming language for complex applications

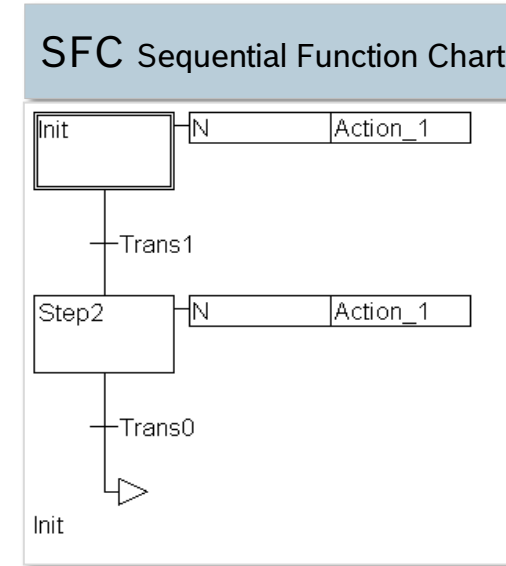
ST Structured Text

```
q1 := i3 AND (i1 OR i2);
```

IndraMotion MLC Programming languages

SFC Sequential Function Chart

- Graphical language
- Program steps are represented by diagram like a flow chart
- Suitable for rapid diagnosis
- The basic elements are steps with action blocks and transitions
- A step consists of a piece of program which is executed as long as the step is active
- As soon as the transition is true the next step becomes active
- Each step can be programmed in any IEC language
- Based on the French Grafcet (IEC 848)



IndraMotion MLC Programming languages

- Programming with ST

IndraMotion MLC Programming languages

ST – Structured Text

- ST is a high-level language
- ST is very similar to PASCAL
- High-level constructs facilitate the structured programming:
 - IF-THEN-ELSE
 - CASE
 - WHILE
 - REPEAT ... UNTIL
 - FOR
 - etc.
- Programmers experienced in other high-level languages like C or PASCAL are immediately familiar with Structured Text

IndraMotion MLC Programming languages

Language Constructs in ST

Key word	Designation	Example	Explanation
:=	Assignment	rDiameter := 12.5;	The variable on the left hand side gets a new value based on the term on the right hand side
	Call of FBs	fbExample(parameter1 := 7, parameter2 := 13);	Call of a function block with input and/or output parameters
IF	Conditional execution	IF d < e THEN f := 1; END_IF;	Execution of alternative blocks of instructions depending on a Boolean expression
CASE	Conditional execution	CASE f OF 1: g := 11; 2: g := 17; ELSE g := 327; END_CASE;	Execution of alternative blocks of instructions based on the value of a control variable (or an integer expression)
FOR	Loop	FOR i := 1 TO 10 DO f[i] := h * 3; END_FOR;	Repetitive execution of a block of instructions with start and end condition
WHILE	Loop	WHILE m > 1 DO n := n / 2; END_WHILE;	Repetitive execution of a block of instructions with start condition
REPEAT	Loop	REPEAT i := i * j; UNTIL i < 100 END_REPEAT;	Repetitive execution of a block of instructions with end condition

IndraMotion MLC Programming languages

Assignment

- In ST an assignment looks as follows:

Variable := expression;

```
uiCounter      := 0;
uiCounter      := uiCounter + 1;
rDistance      := SIN(40.0) * rWidth;
rAreaTriangle  := rBase * rHeight / 2;
bGreen         := FALSE;
byMyByte       := 16#F0;
rAngle         := rAngle MOD 360;
tPeriode       := T#20ms;
```

- The statements are executed from right to left
- In complex terms the operator precedence rules must be considered!

IndraMotion MLC Programming languages

Conditional Execution with IF

```
IF a < b THEN                (* Conditional execution *)  
    c := 0;  
END_IF;
```

```
IF a < b THEN                (* Conditional execution *)  
    c := 0;  
ELSE                          (* with default condition *)  
    c := 1;  
END_IF;
```

```
IF a < b THEN                (* Conditional execution *)  
    c := 0;  
ELSIF a = b THEN             (* with alternate condition *)  
    c := 100;  
ELSE                          (* with default condition *)  
    c := 1;  
END_IF;
```

IndraMotion MLC Programming languages

Conditional Execution with IF

```
IF expression THEN      (* Execution of following block if expression is TRUE *)
instruction_list;

ELSIF expression THEN  (* Execution of following block if preceding      *)
instruction_list;      (* conditions are FALSE and this expression is      *)
                      (* TRUE. Multiple repetitions of this construct  *)
                      (* are supported.                                *)

ELSE                    (* Execution of following block if preceding      *)
instruction_list;        (* conditions FALSE. This element can be omitted. *)

END_IF;                 (* Closing parenthesis of IF statement      *)
```

IndraMotion MLC Programming languages

Conditional Execution with CASE

```
CASE a OF                                     (* depending on the current value of 'a', *)
    1:      c := 1; (* for a unique value of 'a' *)
    2,5,9:  c := 7; (* for 3 concrete values of 'a' → enumeration *)
    6..8:   c := 3; (* for a range of values of 'a' *)
ELSE
    c := 0; (* for all other values of 'a' *)
END_CASE;
```

IndraMotion MLC Programming languages

Conditional Execution with CASE

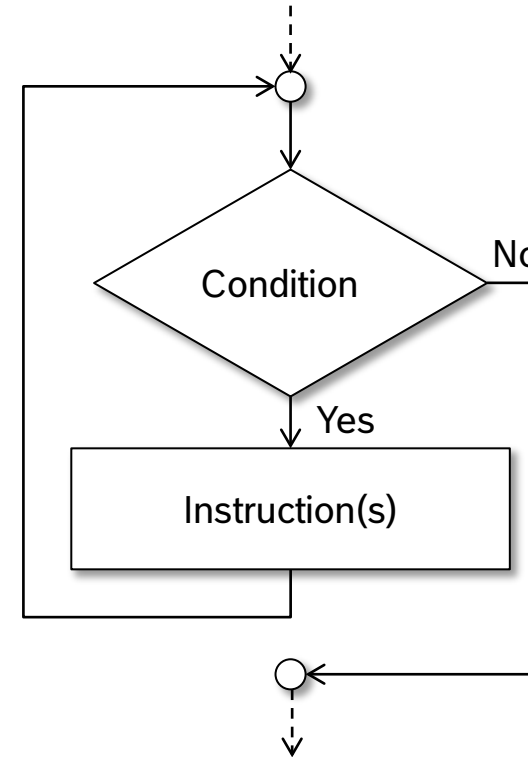
```
CASE expression OF (* expression: integer value (SINT, INT, DINT, LINT) *)
    case_value:
    instruction_list; (* execution if value of expression equals to *)
                      (* case_value. This can be a concrete value, an *)
                      (* enumeration of values, or a range of values. *)
    ELSE              (* execution if expression doesn't equal to any of *)
    instruction_list; (* the listed case_values. This element can be *)
                      (* omitted. *)
END_CASE;            (* Closing parenthesis of CASE statement *)
```

IndraMotion MLC Programming languages

WHILE Loop

```
WHILE valueIn < limit DO  
    counter := counter + 1;  
END_WHILE
```

```
index := 1;  
maximum := 0;  
WHILE index <= 20 DO  
    IF array[index] > maximum THEN  
        maximum := array[index];  
    END_IF  
    index := index + 1;  
END_WHILE
```



IndraMotion MLC Programming languages

REPEAT-UNTIL Loop

REPEAT

```
counter := counter + 1;
```

UNTIL valueIn > 365

END_REPEAT

```
index := 1;
```

```
maximum := 0;
```

REPEAT

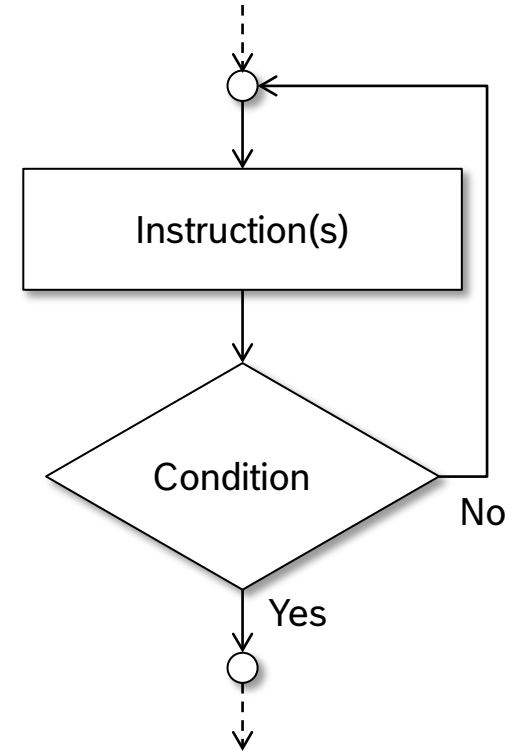
```
    IF array[index] > maximum THEN  
        maximum := array[index];
```

```
    END_IF
```

```
    index := index + 1;
```

UNTIL index > 20

END_REPEAT



IndraMotion MLC Programming languages

FOR Loop

```
FOR index := 2 TO indexMax BY 2 DO
```

```
    counter := counter + 1;
```

```
END_FOR
```

(* If no increment is specified (BY is missing), a default increment value of 1 is used! *)

```
maximum := 0;
```

```
FOR index := 1 TO 20 DO
```

```
    IF array[index] > maximum THEN
```

```
        maximum := array[index];
```

```
    END_IF
```

```
END_FOR
```

IndraMotion MLC Programming languages

- Programming with SFC

Next time!

IndraMotion MLC Programming languages

